CHAPTER 1

INTRODUCTION

Communication between institutions and students is important for sharing information such as event announcements, exam schedules, and academic updates. Traditionally, this information is displayed on physical notice boards on campus. However, these notice boards require manual maintenance and outdated notices may remain if they are not removed on time. To improve accessibility, many institutions have started using digital notice systems.

However, many digital notice systems still lack automated lifecycle management and targeted notification delivery. The Smart Digital Notice Board Lifecycle Manager addresses these issues by using MongoDB with TTL indexes to automatically remove expired notices and FastAPI to manage backend communication. This ensures that only active notices remain visible and users receive relevant notifications.

1.1 Motivation

In educational institutions, notice boards are used to share important announcements with students and staff. However, traditional notice boards and many digital systems still require manual maintenance. Expired notices may remain visible for a long time, and students sometimes receive announcements that are not relevant to them. This creates confusion and increases administrative effort.

- Traditional notice boards require manual updates and maintenance.
- Expired notices often remain displayed for a long time.
- Students may miss important announcements.
- Many digital systems send notifications to all users instead of relevant groups.
- The system uses **MongoDB** TTL indexes to automatically remove expired notices.
- It improves communication by providing relevant and timely information to users.

1.2 Existing System

Many educational institutions use digital notice board systems to share announcements with students and staff. These systems allow administrators to post notices online so that users can access them through institutional portals. However, most existing systems lack automation in managing notice expiry and often require manual maintenance.

• **Fedena Digital Notice Board**

- Allows digital posting and viewing of notices.
- Does not support automatic notice expiry.
- Lacks lifecycle management and personalized notifications.

• **DreamIndia ERP Notice Module**

- Provides an ERP-based announcement system.
- Requires manual deletion of expired notices.
- Sends broadcast notifications to all users.

• **GEGO K12 Notice Board**

- Used for school-level digital notice management.
- Does not provide database-level automation.
- Outdated notices may remain visible in the system.

These limitations highlight the need for a system that can automatically manage notice expiry and deliver relevant notifications to users.

1.3 Problem Statement

Design and develop a MongoDB based digital notice system that automatically manages the notice lifecycle using TTL indexes and provides category-based auto-notifications to users

CHAPTER 2

SOFTWARE STACK

The software stack refers to the technologies used to develop and run the Smart Digital Notice Board system. It includes front-end technologies for the user interface and back-end technologies for handling system logic, data storage, and communication with the database.

2.1 Front End

The front end is the part of the system that users interact with directly. It allows students and administrators to view notices, read announcements, and interact with the system through a simple web interface. The front end communicates with the backend to display the latest notices.

Front-End Components:

- **HTML** – Used to create the structure and layout of the web pages.
- **CSS** – Used to design and style the user interface to make it visually clear and organized.
- **JavaScript** – Used to add interactivity and handle user actions such as loading notices and sending requests to the backend.
- Displays notices and allows users to easily access announcements.

2.2 Back End

The back end handles the core functionality of the system such as storing notices, processing user requests, and managing the notice lifecycle. It communicates with the database to store and retrieve notice information.

Back-End Components

- **FastAPI**

  FastAPI is used as the backend framework to build the server-side application. It handles API requests from the front end and processes operations such as adding notices, retrieving notices, and managing system interactions. FastAPI is known for its high performance and efficiency in building modern web APIs.

- **MongoDB**

  MongoDB is used as the database for storing notice information. It stores data in a document-based format, which allows flexible and efficient storage of notice details such as title, content, category, department, and expiry time. MongoDB is well suited for applications that require scalable and flexible data management.

- **TTL (Time-To-Live) Indexes in MongoDB**

  TTL indexes are used to automatically remove expired notices from the database. Each notice is stored with an expiry time, and the TTL index ensures that the notice is automatically deleted once the expiry time is reached. This eliminates the need for manual deletion of outdated notices.

- **Notice Management and Notification Handling**

  The backend manages the complete lifecycle of notices, including notice creation, storage, retrieval, and expiry management. It also processes notification delivery so that relevant users receive updates when a new notice is posted.

CHAPTER 3

DESIGN

The design of the Smart Digital Notice Board system defines how different components interact to manage notices and deliver notifications to users. The system follows a layered architecture that separates the user interface, backend processing, database management, and notification services. This structure improves scalability, maintainability, and efficient data handling.

3.1. Project Architecture

The project architecture describes how different components of the Smart Digital Notice Board system interact with each other to manage notices and deliver notifications efficiently.

The system consists of **a user layer, frontend interface, backend server, database, and notification services.** The frontend allows administrators to post notices and students to view announcements. The backend processes requests and communicates with the database where notices are stored. Notification services send alerts to students when new notices are posted.

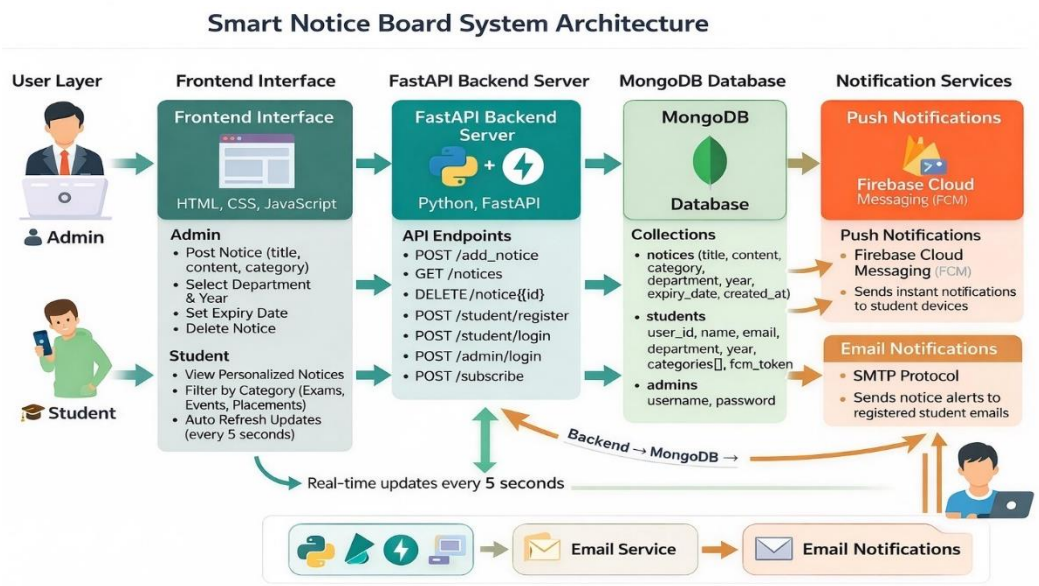


Figure 3.1: Smart Notice Board System Architecture

The above figure illustrates the architecture of the Smart Digital Notice Board system. It shows how the frontend communicates with the backend server, which interacts with the database and notification services to manage notices and deliver updates to users.

3.2 Sequence Diagram

This section details the hardware needed for development and testing of the platform, from high-performance systems for 3D content creation to mobile devices for AR deployment.

A sequence diagram is used to represent the sequence of interactions between different components of the Smart Digital Notice Board system. It shows how requests are sent from the user interface to the backend server and how the system processes those requests step by step.

The sequence diagram illustrates how an administrator creates a notice through the frontend interface, how the backend processes the request, and how the notice is stored in the database. It also shows how students request and retrieve notices from the system and how notifications are delivered when a new notice is posted.

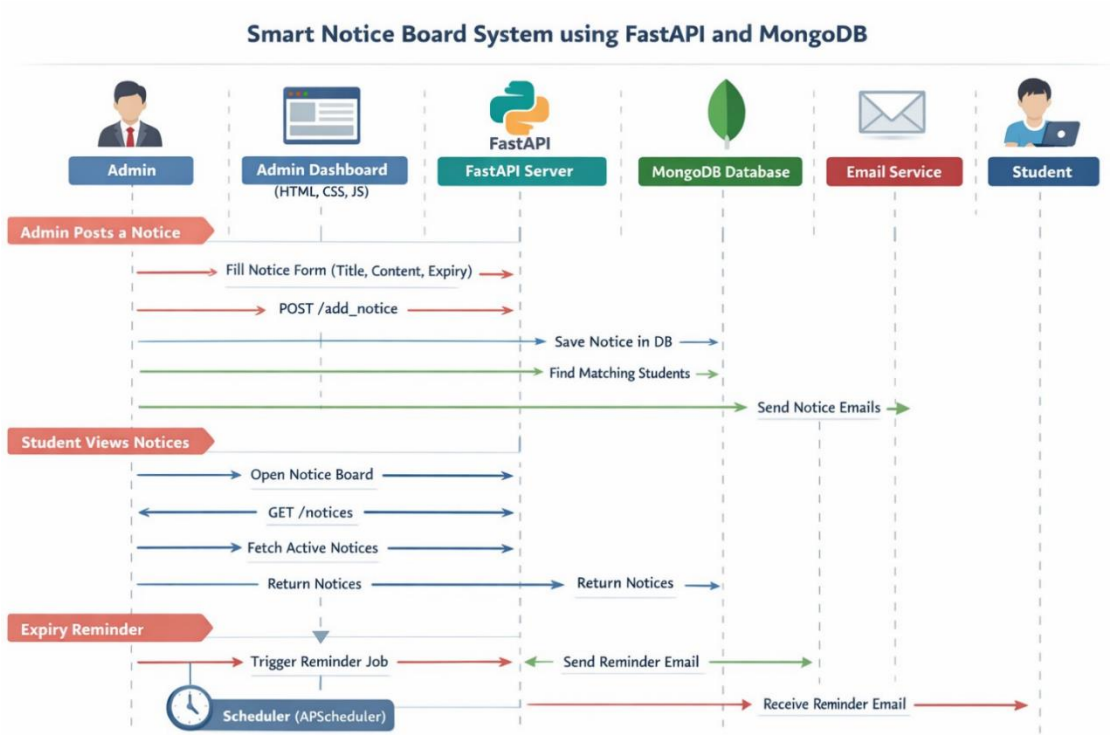


Figure 3.2: Sequence Diagram of Smart Digital Notice Board System

The above figure shows the sequence of interactions between the user interface, backend server, and database during the notice management process. It explains how notices are created, stored, and retrieved by the system and how users interact with the application to access announcements.

CHAPTER-4

METHODOLOGY AND IMPLEMENTATION

This chapter outlines the step-by-step methodology and implementation strategy followed in the AR Explorers project. It details the planning, design, and integration phases involved in building an interactive WebAR-based learning experience. From 3D model creation to user interaction, quizzes, and feedback collection, each component is explained in a structured manner.

4.1 Pseudocode

This section presents the pseudocodes used to describe the major operations of the Smart Digital Notice Board Lifecycle Manager. Pseudocode helps in understanding the logical flow of the system without depending on a specific programming language. The following pseudocodes explain the processes involved in student registration, notice creation, notice retrieval, and automatic deletion of expired notices.

4.1.1 Pseudocode for Student Registration

This pseudocode explains the process through which a student registers in the system. The user provides the required information such as name, email, department, and category preferences. After validation, the student details are stored in the database for further use.

```
BEGIN

DISPLAY registration form

INPUT name, email, department, year, category

IF input data is valid THEN
CREATE student_record
STORE student_record in database
DISPLAY "Registration Successful"
ELSE
DISPLAY "Invalid Input"
END IF

END
```

Figure 4.1: Student Registration

The above pseudocode describes the student registration process in the system. It ensures that only valid student information is stored in the database and allows users to register successfully in the Smart Digital Notice Board system.

4.1.2 Pseudocode for Notice Creation

This pseudocode describes how the administrator creates and publishes a new notice in the system. The administrator enters details such as notice title, content, category, department, and expiry time. After validation, the notice is stored in the database.

```
ADMIN login

IF authentication successful THEN
DISPLAY notice creation form
INPUT title, content, category, department, expiry_time
CREATE notice_record
STORE notice_record in database
DISPLAY "Notice Added Successfully"
ELSE
DISPLAY "Login Failed"
END IF

END
```

Figure 4.2: Notice Creation

The above pseudocode explains how administrators manage notices in the system. It ensures that only authorized users can create and publish notices, which are then stored in the database.

4.1.3 Pseudocode for Notice Retrieval

This pseudocode explains how the system retrieves and displays notices to students. When a user requests to view notices, the system fetches the relevant notices from the database and displays them on the interface.

```
USER requests notices

FETCH notices from database

FILTER active notices

FOR each notice DO
DISPLAY notice details
END FOR

END
```

Figure 4.3: Notice Retrieval

The above pseudocode describes how the system retrieves notices stored in the database and displays them to users. This process ensures that students can easily access important announcements and updates.

4.1.4 Pseudocode for Automatic Notice Expiry

This pseudocode explains how expired notices are automatically removed from the system. The system uses a TTL (Time-To-Live) mechanism to delete notices once their expiry time is reached.

```
BEGIN

SET TTL index on expiry_time

WHILE system running DO
IF current_time ≥ notice.expiry_time THEN
DELETE notice from database
END IF
END WHILE

END
```

Figure 4.4: Automatic Notice Expiry

The above pseudocode describes the automatic notice lifecycle management process. Expired notices are automatically deleted from the database, ensuring that only valid and active notices remain in the system.

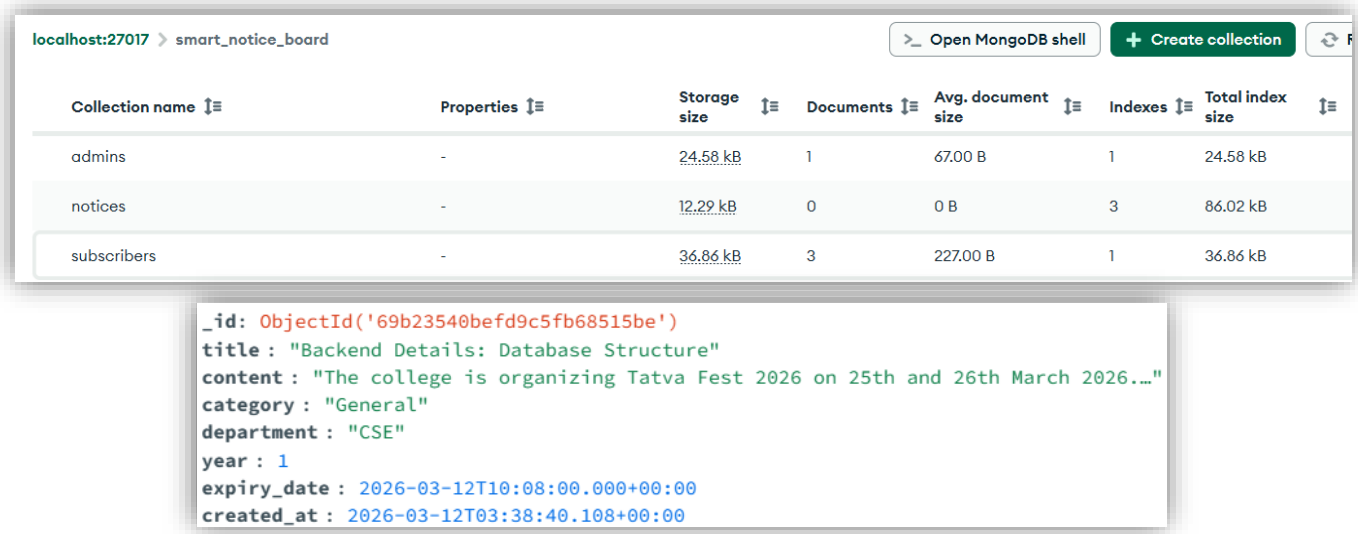
4.2 Database

The Smart Notice Board system uses MongoDB as its backend database to store and manage application data. MongoDB is a document-oriented NoSQL database that stores data in JSON-like documents, allowing flexible and scalable data management. This approach eliminates the rigid schema structure used in relational databases and enables easy modification of records when application requirements evolve.

In this system, the database stores information related to notices, student subscribers, and administrators. Each type of data is organized into separate collections to ensure logical separation and efficient data retrieval. The collections used in the system are Notices, Subscribers, and Admins. These collections allow the system to manage notice publishing, student subscriptions, and administrator authentication effectively.

4.2.1 MongoDB Collections in Smart Notice Board Database

The Notices collection stores all the notices posted by the administrator. Each notice contains details such as the title, content, category, department, academic year, expiry date, and creation timestamp. This information helps in targeting notices to specific groups of students based on department, year, or category. Additionally, the expiry date enables automatic filtering of outdated notices so that only active notices are displayed to users.



Collection name	Properties	Storage size	Documents	Avg. document size	Indexes	Total index size
admins	-	24.58 kB	1	67.00 B	1	24.58 kB
notices	-	12.29 kB	0	0 B	3	86.02 kB
subscribers	-	36.86 kB	3	227.00 B	1	36.86 kB

Figure 4.5: MongoDB Collections in Smart Notice Board Database

This figure shows the collections present in the Smart Notice Board database, including admins, notices, and subscribers. It illustrates how MongoDB organizes application data into separate collections for efficient storage and retrieval.

4.2.2 Admin Collection Document Structure

The **Admins collection** stores administrator credentials used for authentication. Each admin document contains a username and password, which allows authorized users to log into the admin dashboard and manage notices. By separating admin credentials from student records, the system maintains a clear distinction between administrative and user roles.

```
_id: ObjectId('69981c2d2485ee44e207204f')
username : "Srusti"
password : "srusti123"
```

Figure 4.6: Admin Collection Document Structure

This figure displays a sample document from the admin collection containing the administrator's username and password. It demonstrates how administrator authentication data is stored securely in the database.

4.2.3 Subscriber Collection Document Structure

The **Subscribers collection** maintains the records of students who register in the system. Each document includes the student's unique ID, name, email address, password, department, academic year, and the categories of notices they have subscribed to. This information is used by the backend to determine which students should receive email notifications when a relevant notice is published.

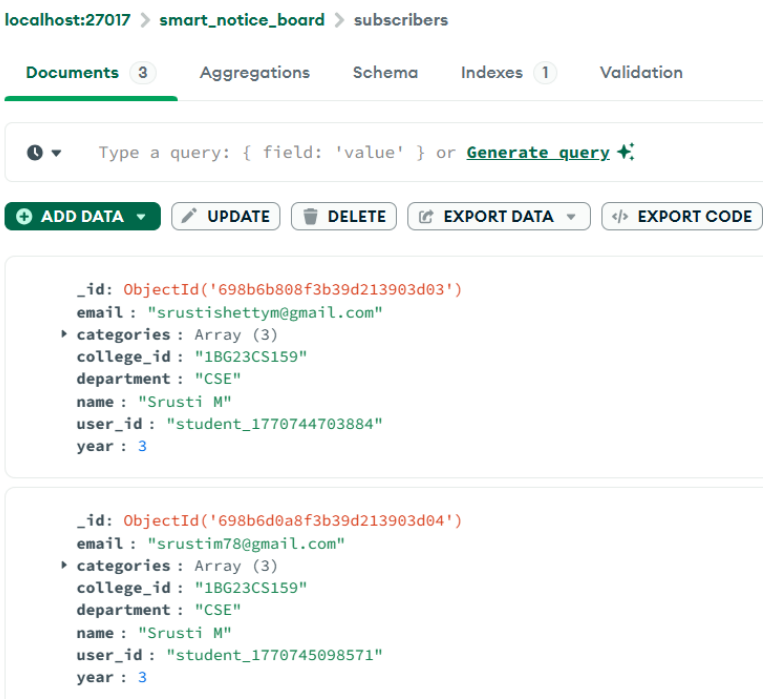


Figure 4.7: Subscriber Collection Document Structure

This figure shows a sample subscriber record containing student details such as user ID, email, department, year, and subscribed categories. It demonstrates how the system stores student preferences to send relevant notice notifications.

Using MongoDB provides several advantages such as scalability, flexibility in data modeling, and efficient querying of large datasets. Since the system stores documents instead of relational tables, it becomes easier to add new attributes or modify existing ones without restructuring the entire database.

4.3 Code Snippets.

This section presents selected code snippets from the Smart Notice Board system along with brief explanations of their functionality. These snippets represent the core components of the backend logic responsible for user registration, notice publishing, and automated notification scheduling. The system is implemented using FastAPI for backend development and MongoDB for storing application data. Each code segment demonstrates how different modules of the system interact with the database and handle user requests efficiently.

The selected snippets highlight important operations such as registering students in the system, publishing notices by administrators, sending notifications to targeted students, and scheduling reminder emails before a notice expires. These functions ensure that the notice management process is automated, organized, and capable of delivering timely information to the intended recipients.

4.3.1 Student Registration Module

The student registration module allows new users to create an account in the Smart Notice Board system. When a student submits their registration details, the system verifies whether the user already exists in the database using their unique user ID. If the user is not present, the system stores the student's information, including name, email, department, academic year, and preferred notice categories, in the subscriber's collection.

This information plays an important role in enabling targeted notifications. By storing student preferences and academic details, the system can later filter notices and deliver relevant updates only to the appropriate group of students. This improves communication efficiency and prevents students from receiving irrelevant announcements.

```
@app.post("/student/register")
def student_register(student: Student):

    existing = subscribers_collection.find_one({"user_id": student.user_id})

    if existing:
        raise HTTPException(status_code=400, detail="User already exists")

    subscribers_collection.insert_one(student.dict())

    return {"message": "Student registered successfully"}
```

4.3.2 Add Notice and Send Notifications

The add notice module represents the central functionality of the Smart Notice Board system. When an administrator posts a new notice, the notice details such as title, content, category, department, year, and expiry date are stored in the database. Once the notice is successfully saved, the system searches the subscribers collection to identify students whose department, academic year, and subscribed categories match the notice criteria.

After identifying the appropriate recipients, the system automatically sends email notifications to those students. This ensures that important announcements reach the intended audience immediately after they are published. By automating this process, the system eliminates manual distribution of notices and improves communication efficiency within the institution.

```
@app.post("/add_notice")
def add_notice(notice: Notice):

    created_time = datetime.utcnow()

    data = notice.dict()
    data["created_at"] = created_time

    notices_collection.insert_one(data)

    students = list(subscribers_collection.find({
        "department": notice.department,
        "year": notice.year,
        "categories": notice.category
    }))

    for student in students:
        send_email(
            to_email=student["email"],
            subject=notice.title,
            body=notice.content
        )

    return {"message": "Notice added and emails sent"}
```

4.3.3 Reminder Notification Scheduler

The reminder notification scheduler is responsible for sending automated reminder emails before a notice expires. After a notice is created, the system calculates a reminder time based on the notice's expiry date. Using a background scheduling mechanism, the system triggers a reminder notification to all relevant students before the notice expires.

This feature ensures that students receive a final reminder about important notices that may require their attention. By notifying users before expiration, the system helps prevent missed deadlines or overlooked announcements. The scheduler runs in the background without interrupting the main application, enabling continuous and efficient operation of the notice board system.

```
def send_expiry_reminder(notice_data, students):
    for student in students:
        send_email(
            to_email=student["email"],
            subject=f"Reminder: {notice_data['title']} expires soon",
            body=f"{notice_data['content']}\n\nThis notice will expire in 24 hours."
        )
```

Chapter 5

RESULTS

This chapter presents the results obtained after implementing the Smart Digital Notice Board Lifecycle Manager. The system was tested to verify the functionality of notice creation, storage, retrieval, and automatic deletion using NoSQL database features. The results confirm that the system successfully performs automated notice lifecycle management.

5.1 Snapshots

This section presents the screenshots of different modules of the Smart Digital Notice Board system. These snapshots illustrate how the system operates and demonstrate the interaction between the user interface, backend services, and the database.

The snapshots provide visual confirmation of the successful implementation of the system and highlight the key functionalities such as notice creation, notice display, and database storage. These screenshots help in understanding how the system manages notices and ensures efficient information delivery to users.

5.1.1 Student View

The student view represents the interface through which students interact with the Smart Digital Notice Board system. Through this interface, students can register in the system and access the platform using their credentials. After logging in, students are able to view the notices that are published by the administrator. This ensures that all important announcements and updates are easily accessible to them in one centralized platform.

This module helps students stay informed about academic activities, examination schedules, events, placement updates, and other important institutional notifications. The interface is designed to be simple, clear, and user-friendly so that students can quickly read and understand the notices without difficulty. By providing organized and timely information, the student view improves communication between the administration and students, ensuring that no important information is missed.

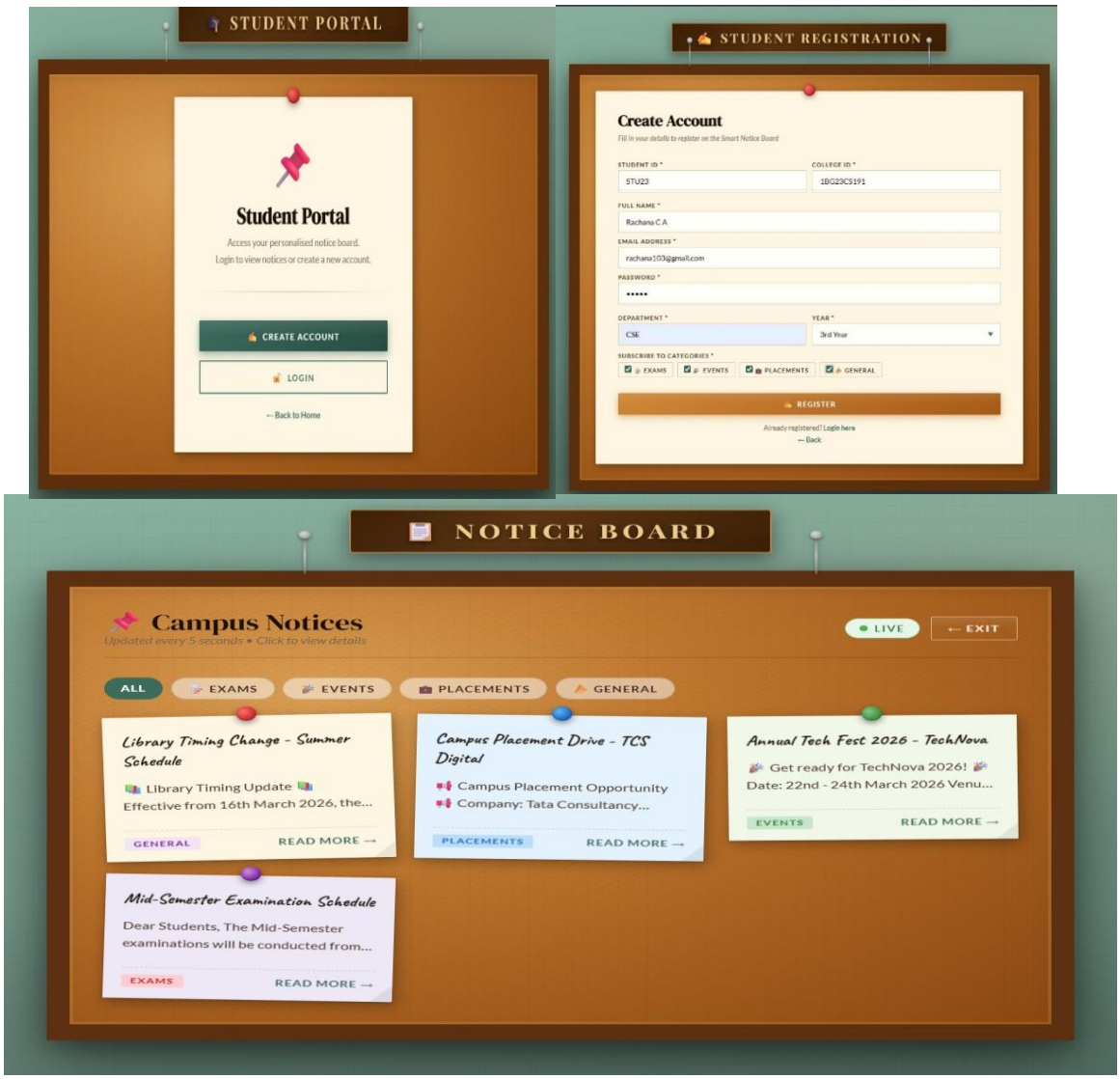


Figure 5.1: Student Interface (Registration and Notice View)

The above figure shows the student interface of the system. It includes the student registration page where users provide their details to join the system and the notice viewing page where students can view the notices that are published by the administrator.

5.1.2 Administrator View

The administrator view provides the management interface of the system. Through this module, authorized administrators can log in to the system and manage notices. The administrator can create and publish notices that will be displayed to the students.

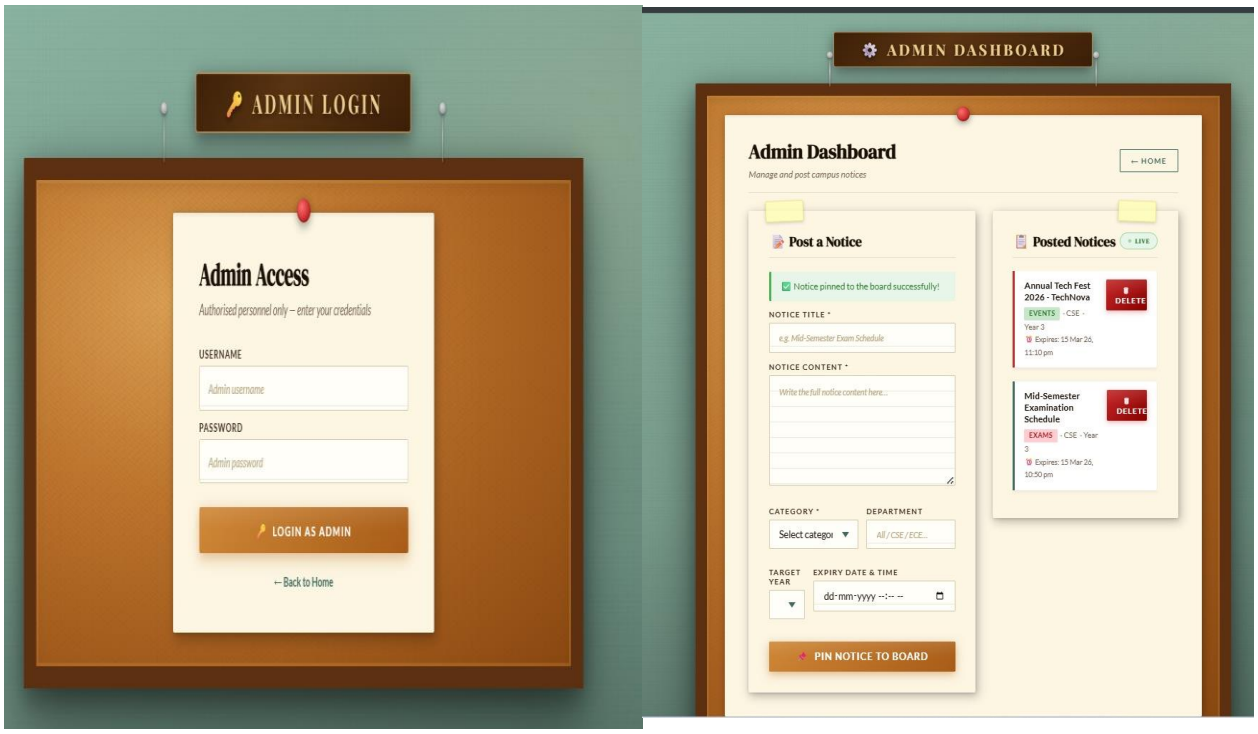
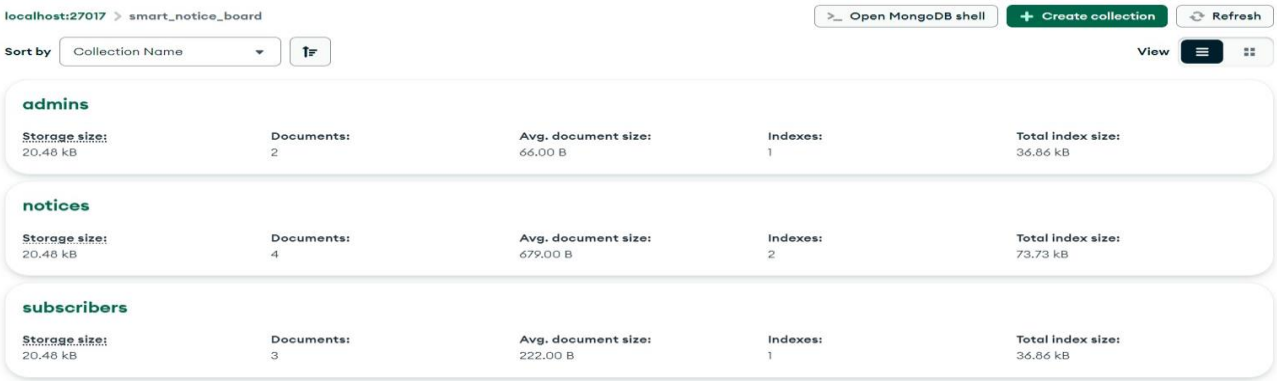


Figure 5.2: Administrator Interface (Login and Notice Management)

The above figure shows the administrator interface of the Smart Digital Notice Board system. It includes the administrator login page and the notice management interface where the administrator can create and publish new notices by providing details such as title, content, category, and expiry time.

5.1.3 Database Storage View

The database storage view illustrates how the information related to students, notices, and administrators is stored in the database. The system uses MongoDB to store data in the form of documents organized into different collections.



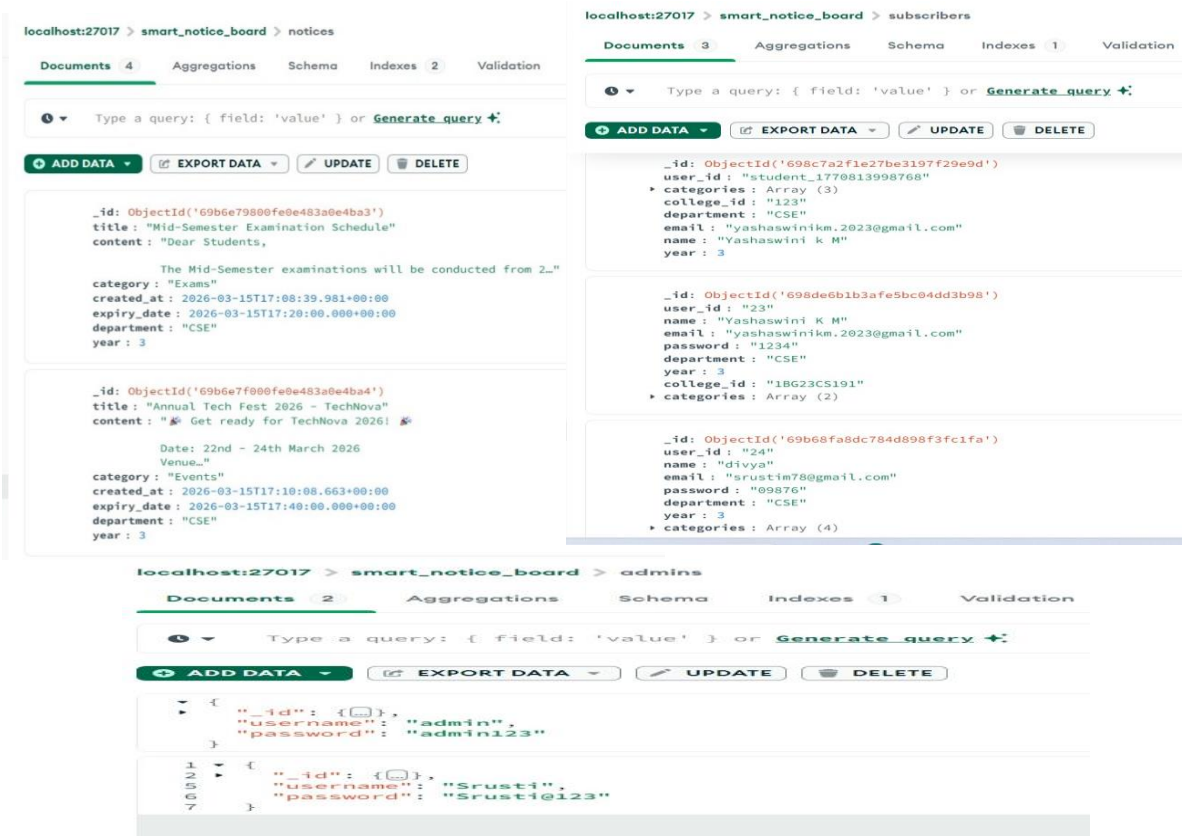


Figure 5.3: Database Collections (Student, Notice, and Administrator)

The above figure shows the database collections used in the system. It includes the student collection, notice collection, and administrator collection where the respective data is stored and managed within the MongoDB database.

5.2 Testing

Testing was conducted to verify that the Smart Digital Notice Board system functions correctly in different scenarios. The testing process focused on validating notice creation, database storage, and automatic deletion of expired notices using TTL indexes. The following tests demonstrate the proper functioning of the system components.

5.2.1 Notice Creation API Test

This test verifies whether the backend API successfully creates a new notice and stores it in the database. The administrator sends a request to the backend service, and the system processes the request and saves the notice details in the database

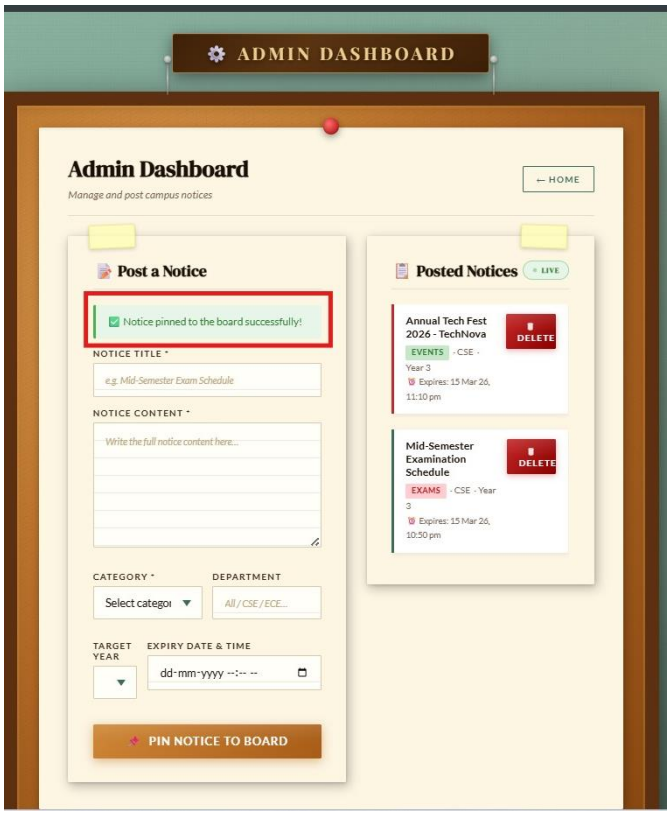


Figure 5.4: Notice Creation API Test

The above figure shows the successful execution of the API used to create a new notice in the system. The response confirms that the notice has been successfully processed and stored in the database through the backend service.

5.2.2 Database Storage Verification

This test verifies that the notice information is correctly stored in the database after it has been created. The database collection is checked to confirm that the notice document contains all the required fields such as title, content, category, department, targeted year, creation time, and expiry time. Proper verification ensures that the system successfully saves the notice details without any data loss or corruption. This step is important because the stored data will later be used to display notices to students and send notifications based on the specified criteria.

Additionally, the stored notice document is examined to ensure that the values entered by the administrator match the data saved in the database. The timestamp fields such as creation time and expiry time are also validated to confirm that the notice will remain active only for the defined duration. This verification process helps ensure that the database operations, including insertion and retrieval of notice records, are functioning correctly.

ADD DATA | EXPORT DATA | UPDATE | DELETE


```
_id: ObjectId('69b6e79800fe0e483a0e4ba3')
title : "Mid-Semester Examination Schedule"
content : "Dear Students,

          The Mid-Semester examinations will be conducted from 2…"
category : "Exams"
created_at : 2026-03-15T17:08:39.981+00:00
expiry_date : 2026-03-15T17:20:00.000+00:00
department : "CSE"
year : 3
```

```
_id: ObjectId('69b6e7f000fe0e483a0e4ba4')
title : "Annual Tech Fest 2026 - TechNova"
content : "🎉 Get ready for TechNova 2026! 🎉

          Date: 22nd - 24th March 2026
          Venue…"
category : "Events"
created_at : 2026-03-15T17:10:08.663+00:00
expiry_date : 2026-03-15T17:40:00.000+00:00
department : "CSE"
year : 3
```


Figure 5.5: Database Storage Verification

The above figure shows the notice document stored in the **MongoDB** database. It confirms that the notice information has been successfully saved in document format and can be retrieved when required.

5.2.3 TTL Index Expiry Test

This test verifies the automatic deletion of expired notices using the TTL index feature. When the expiry time of a notice is reached, the database automatically removes the document without requiring manual intervention.

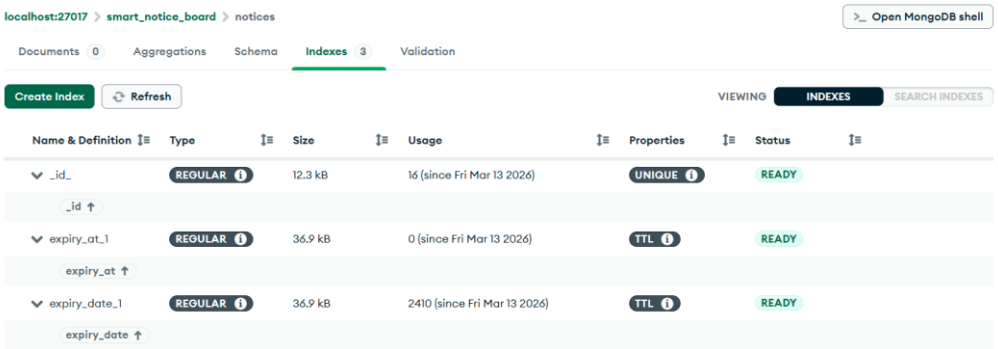


Figure 5.6: TTL Index Expiry Test

The above figure demonstrates the TTL index functionality where expired notices are automatically deleted from the database. This confirms that the system successfully implements automated notice lifecycle management using NoSQL database features.

5.2.4 Notice Retrieval and Display Test

This test verifies whether the system correctly retrieves notices from the database and displays them to users through the interface. The system fetches active notices from the database and presents them in a structured format so that users can easily view important announcements. The retrieved notices include essential details such as the title, category, department, target year, and expiry time. This ensures that students and users can quickly identify notices that are relevant to them without confusion.

In addition, the test checks whether only valid and active notices are displayed while expired notices are filtered out automatically. The interface is examined to confirm that the notices are organized clearly and updated whenever new notices are added by the administrator. This verification also ensures that the connection between the frontend interface and the database is functioning properly. Successful completion of this test confirms that the system can reliably retrieve stored notices and display them accurately to users.

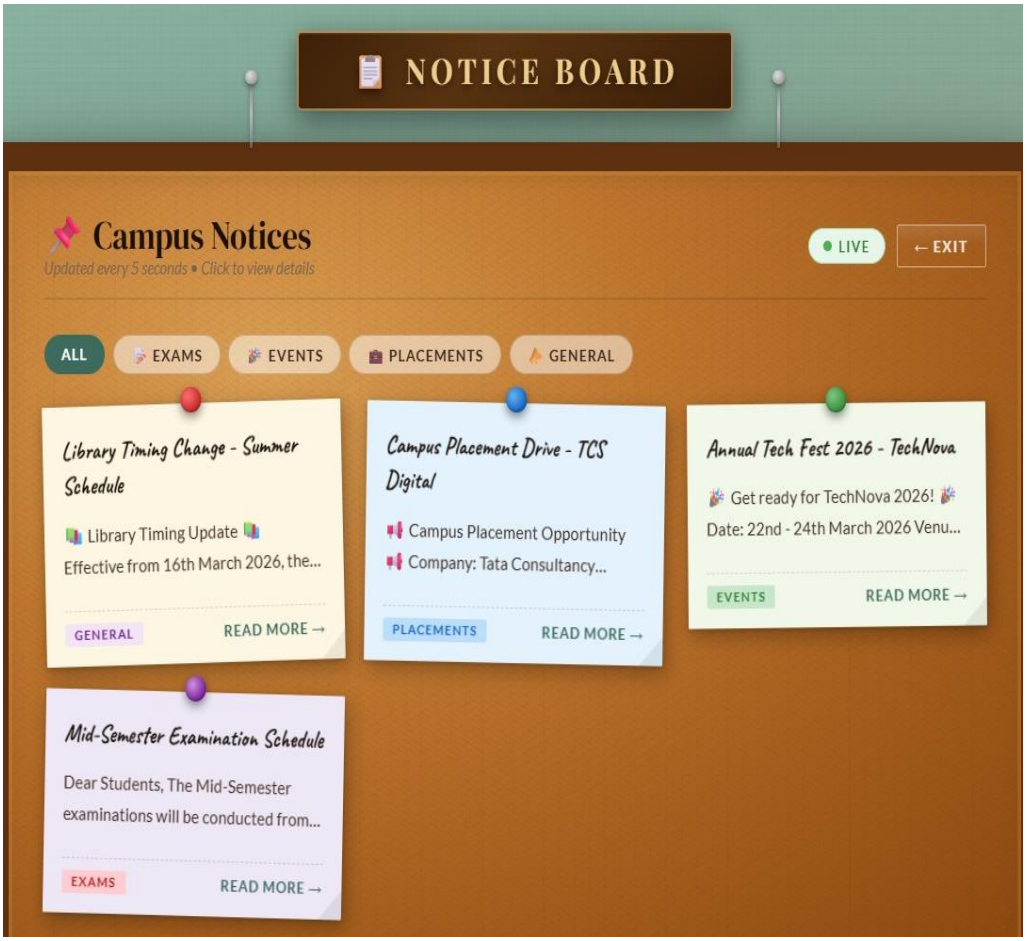


Figure 5.7: Notice Retrieval and Display Test

The above figure shows the notice display interface where users can view active notices. The system retrieves the notice data from **MongoDB** and displays it through the frontend interface, ensuring that users receive updated information.

CHAPTER-6

CONCLUSION

The Smart Digital Notice Board Lifecycle Manager demonstrates the practical use of NoSQL technologies in managing institutional notices efficiently. By using a document-based database, the system provides flexible data storage and automated lifecycle management. The project shows how NoSQL databases can be applied to real-world applications that require scalability, dynamic data structures, and automated data handling. Future Enhancements

6.1 Learning Outcomes

This project helped in understanding how NoSQL databases can be used in real-world applications. It also provided practical knowledge about integrating a NoSQL database with backend technologies to build a functional web system.

- Understanding the working of **MongoDB** as a document-based NoSQL database.
- Learning flexible schema design for storing notice information.
- Implementing TTL (Time-To-Live) indexes for automatic deletion of expired notices.
- Understanding category-based data filtering using NoSQL queries.
- Developing APIs using **FastAPI** for database communication.
- Learning how NoSQL databases support scalable data management.

6.2 Advantages

The system provides several advantages by using NoSQL database features and automated lifecycle management. It improves the efficiency of notice management and reduces the need for manual maintenance.

Advantages:

- Flexible document-based data storage.
- Automatic deletion of expired notices using TTL indexes.
- Efficient retrieval of notices based on categories.
- Reduced manual effort for administrators.
- Faster data handling and scalability.
- Improved notice management system.

6.3 Future Scope

The project can be extended further by adding more advanced features and improving system scalability. Future improvements can enhance both the functionality and usability of the system.

Future Scope:

- Integration with mobile applications for better accessibility.
- Implementation of real-time notification systems.
- Expansion to support multiple institutions.
- Use of distributed NoSQL databases for higher scalability.
- Adding analytics for better notice management.

REFERENCES

[1] K. Chodorow, *MongoDB: The Definitive Guide*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2013.

[2] S. Tiwari, *Professional NoSQL*. Indianapolis, IN, USA: Wrox Press, 2011.

[3] R. Cattell, "Scalable SQL and NoSQL Data Stores," *ACM SIGMOD Record*, vol. 39, no. 4, pp. 12–27, 2011.

[4] Documentation of **MongoDB**, MongoDB Inc. [Online]. Available: https://www.mongodb.com/docs/

[5] Introduction to **NoSQL databases**, MongoDB Inc. [Online].
Available: https://www.mongodb.com/nosql-explained

[6] Documentation of **FastAPI**. [Online]. Available: https://fastapi.tiangolo.com/

[7] MDN Web Docs, HTML, CSS and JavaScript Documentation. [Online]. Available: https://developer.mozilla.org/